Analysis of Unstructured Data

Lecture 2 — Data Acquisition

Janusz Szwabiński

> How we acquire data fixes its **coverage**, its **bias**, its **legality**, and whether the result is **reproducible**!

Outlook

1. Acquisition in miniature: HTML to records (motivation)
2. Mathematical foundations
3. Sources in practice
4. Cross-cutting concerns (licensing, PII, provenance and caching)

Further reading

1. Manning, *Introduction to Information Retrieval* (2009)
2. Gebru et al., *Datasheets for Datasets* (2021), https://arxiv.org/pdf/1803.09010
3. Bender & Friedman, *Data Statements for NLP* (2018)
4. Horvitz & Thompson, A Generalization of Sampling Without Replacement from a Finite Universe, JASA 47 (1952)
5. Särndal, Swensson & Wretman, Model Assisted Survey Sampling (1992)
6. `requests`, Beautiful Soup and Hugging Face datasets documentation

1. Acquisition in miniature: HTML to records

- Most unstructured sources arrive wrapped in markup or a document format.
- Acquisition is the act of *imposing a schema* on them.

Here is the smallest complete example: a fragment of an HTML listing page parsed into a clean table.

In [5]:
```
from bs4 import BeautifulSoup
import pandas as pd

# A synthetic listing page. Real scraping only changes where this string comes f
html = """
<html><body>
  <div class="item"><h2>Alpha sensor</h2><span class="price">12.50</span><a href
  <div class="item"><h2>Beta sensor</h2><span class="price">7.00</span><a href="
  <div class="item"><h2>Gamma sensor</h2><span class="price">19.90</span><a href
</body></html>
"""
```

```
soup = BeautifulSoup(html, "html.parser")
records = [{"name": d.find("h2").get_text(strip=True),
            "price": float(d.find("span", "price").get_text(strip=True)),
            "url": d.find("a")["href"]}
           for d in soup.select("div.item")]

pd.DataFrame(records)
```

Out[5]:

	name	price	url
0	Alpha sensor	12.5	/p/alpha
1	Beta sensor	7.0	/p/beta
2	Gamma sensor	19.9	/p/gamma

2. Mathematical foundations

2.1 Acquisition is sampling - selection bias and its correction

Let $\{1, \dots, N\}$ be a population with a quantity of interest y_i whose mean,

$$\bar{y} = \frac{1}{N} \sum_i y_i,$$

we would like to estimate:

- an API or a crawl does **not** return a uniform sample
- item i is included with some probability π_i, and π_i is usually correlated with y_i (popular, recent or highly-linked items are over-represented)

The sampling model

We regard the population $\{1, \dots, N\}$ and its values y_i as **fixed**; the only randomness is in *which* items end up in our dataset. Describe acquisition by inclusion indicators

$$I_i = \begin{cases} 1, & \text{item } i \text{ is acquired (returned by the API, visited by the crawler)}, \\ 0, & \text{otherwise}, \end{cases}$$

and let $S = \{i : I_i = 1\}$ be the set of items we actually obtain. The number π_i,

$$\pi_i = P(I_i = 1) > 0,$$

is the **inclusion probability** of item i.

The sample size is simply the number of ones among the indicators, so it is a random variable:

$$n = |S| = \sum_{i=1}^{N} I_i.$$

Each I_i is a Bernoulli variable, so $E[I_i] = 1 \cdot \pi_i + 0 \cdot (1 - \pi_i) = \pi_i$, and by **linearity of expectation**

$$E[n] = \sum_{i=1}^{N} E[I_i] = \sum_{i=1}^{N} \pi_i.$$

This step needs no independence: the identity holds for *every* sampling design. (For simple random sampling of n out of N, $\pi_i = n/N$ and the sum is exactly n.)

The simplest design, which we use in the simulation, is **Poisson sampling**: the I_i are *independent* Bernoulli(π_i) variables, as if every item tossed its own biased coin. Independence enters only at the level of the variance, which is then the sum of individual variances:

$$\mathrm{Var}(n) = \sum_{i=1}^{N} \pi_i \, (1 - \pi_i).$$

The Horvitz–Thompson estimator

The idea of Horvitz and Thompson (1952) is to weight each observed item by the inverse of its inclusion probability, i.e. by $w_i = 1/\pi_i$. This is the **inverse probability weighting (IPW)**. An item with $\pi_i = 0.05$ "stands in" for about 20 population units like it.

$$\hat{\bar{y}}_{\mathrm{HT}} = \frac{1}{N} \sum_{i \in S} \frac{y_i}{\pi_i} = \frac{1}{N} \sum_{i=1}^{N} I_i \, \frac{y_i}{\pi_i}.$$

Unbiasedness. The second form sums over the *whole* population, and only the I_i are random. Since $E[I_i] = \pi_i$,

$$E\big[\hat{\bar{y}}_{\mathrm{HT}}\big] = \frac{1}{N} \sum_{i=1}^{N} \frac{E[I_i]}{\pi_i} \, y_i = \frac{1}{N} \sum_{i=1}^{N} y_i = \bar{y}.$$

No assumption on the distribution of y is needed, only **positivity**: $\pi_i > 0$ for every i. An item with $\pi_i = 0$ can never be observed, and no reweighting can recover it.

Variance. Under Poisson sampling, $\mathrm{Var}(I_i) = \pi_i(1 - \pi_i)$ and the I_i are independent, so

$$\mathrm{Var}\big(\hat{\bar{y}}_{\mathrm{HT}}\big) = \frac{1}{N^2} \sum_{i=1}^{N} \frac{1 - \pi_i}{\pi_i} \, y_i^2,$$

with an unbiased estimate computable from the sample alone:

$$\widehat{\mathrm{Var}} = \frac{1}{N^2} \sum_{i \in S} \frac{1 - \pi_i}{\pi_i^2} \, y_i^2.$$

(For designs with dependent inclusions the formula also involves the joint probabilities $\pi_{ij} = P(I_i = I_j = 1)$; the structure of the argument is the same.)

Why π_i^2 in the last formula? The variance is a sum over the **whole population**, $\sum_{i=1}^{N} g_i$ with $g_i = \frac{1-\pi_i}{\pi_i} \, y_i^2$, but we only know g_i for $i \in S$. Estimating a population sum from a

sample is exactly what Horvitz–Thompson does: divide each observed term by its inclusion probability.

Two features of this formula matter in practice:

- small π_i enter through $1/\pi_i$, so **rarely sampled items dominate the variance**;
- the variance depends on y_i^2, not on $(y_i - \bar{y})^2$, so HT is **not invariant to shifting** y by a constant.

The Hájek (self-normalised) estimator

HT divides by the population size N, which in data acquisition is often unknown (how many documents does the site hold?). But N can itself be estimated by HT, applied to $y_i \equiv 1$:

$$\hat{N} = \sum_{i \in S} \frac{1}{\pi_i}, \qquad E[\hat{N}] = N.$$

Replacing N by $\hat{N}$ gives the **Hájek estimator** (1971):

$$\hat{\bar{y}}_{\mathrm{H}} = \frac{\sum_{i \in S} y_i/\pi_i}{\sum_{i \in S} 1/\pi_i}.$$

Its properties:

- it is a ratio of two unbiased estimators, so it has a small bias of order $O(1/n)$, but it is **consistent**;
- it is a weighted mean with weights summing to one, so it is **invariant to shifts** $y_i \mapsto y_i + c$;
- by linearisation, its variance is approximately

$$\operatorname{Var}\big(\hat{\bar{y}}_{\mathrm{H}}\big) \approx \frac{1}{N^2} \sum_{i=1}^{N} \frac{1 - \pi_i}{\pi_i} (y_i - \bar{y})^2,$$

  the HT formula with y_i replaced by the deviations $y_i - \bar{y}$, which is usually much smaller;
- it requires π_i only **up to a multiplicative constant**, because the constant cancels in the ratio. This is valuable when we know only *relative* inclusion probabilities, for example "a page with twice the links is crawled twice as often".

In practice the Hájek form is the default.

In [6]:
```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(1)
N = 100_000
popularity = rng.beta(2, 5, N)                          # skewed toward smal
y = 1.0 + 3.0 * popularity + rng.normal(0, 0.3, N)      # quantity of intere
true_mean = y.mean()

pi = 0.02 + 0.25 * popularity                           # inclusion prob gro
```

```
included = rng.random(N) < pi                                   # Poisson sampling

def estimators(y, pi, included, N):
    ys, ps = y[included], pi[included]
    naive = ys.mean()
    ht = (ys / ps).sum() / N
    hajek = (ys / ps).sum() / (1.0 / ps).sum()
    return naive, ht, hajek

naive, ht, hajek = estimators(y, pi, included, N)
{"n": int(included.sum()), "true_mean": float(true_mean),
 "naive": float(naive), "HT": float(ht), "Hajek": float(hajek)}
```

Out[6]:
```
{'n': 9106,
 'true_mean': 1.8570392762991763,
 'naive': 2.0634697136842894,
 'HT': 1.848817866240964,
 'Hajek': 1.8580191625594331}
```

A single run shows little: bias and variance are properties of the *sampling distribution*. Thus, let us repeat the acquisition many times on the same population and compare the empirical standard deviations with the formulas derived above.

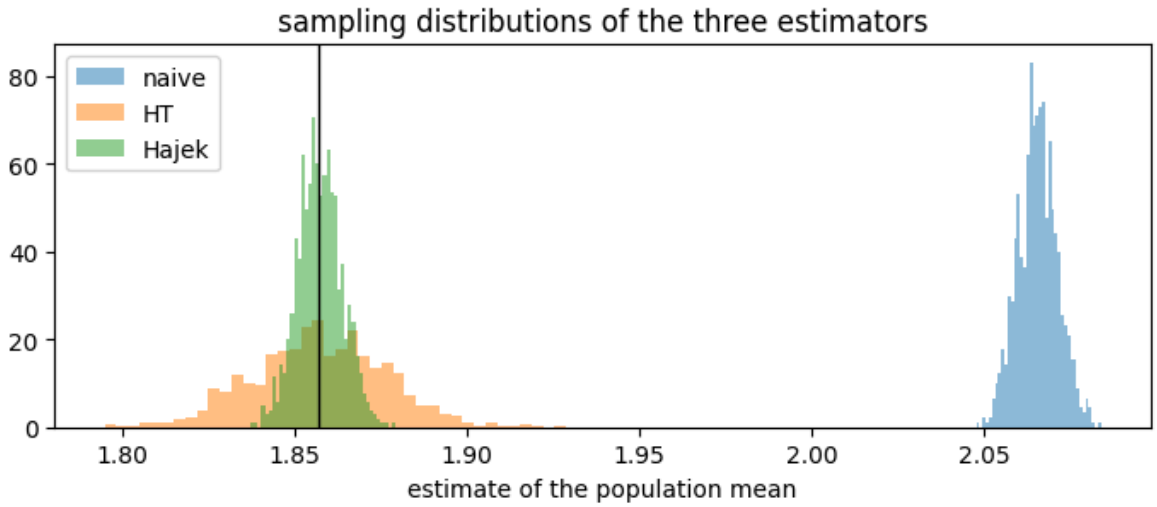
In [7]:
```
R = 1000
est = np.array([estimators(y, pi, rng.random(N) < pi, N) for _ in range(R)])

sd_ht_theory = np.sqrt(((1 - pi) / pi * y**2).sum()) / N
sd_hajek_theory = np.sqrt(((1 - pi) / pi * (y - true_mean)**2).sum()) / N

for name, col, sd_th in zip(["naive", "HT", "Hajek"], est.T,
                            [np.nan, sd_ht_theory, sd_hajek_theory]):
    print(f"{name:6s} bias = {col.mean() - true_mean:+.4f}   "
          f"sd = {col.std(ddof=1):.4f}   sd (theory) = {sd_th:.4f}")

fig, ax = plt.subplots(figsize=(7, 3.2))
for name, col in zip(["naive", "HT", "Hajek"], est.T):
    ax.hist(col, bins=40, density=True, alpha=0.5, label=name)
ax.axvline(true_mean, color="k", lw=1)
ax.set_title("sampling distributions of the three estimators")
ax.set_xlabel("estimate of the population mean"); ax.legend()
plt.tight_layout()
plt.show()
```

```
naive  bias = +0.2086   sd = 0.0057   sd (theory) = nan
HT     bias = +0.0003   sd = 0.0194   sd (theory) = 0.0192
Hajek  bias = +0.0003   sd = 0.0064   sd (theory) = 0.0063
```



Why the Hájek form is preferred: shifting y

Adding a constant to y is a harmless change of units (think °C vs K). The Hájek estimator does not notice it; the variance of HT grows with y^2 and becomes very large.

In [8]:
```
c = 100.0
est_shift = np.array([estimators(y + c, pi, rng.random(N) < pi, N) for _ in rang
dict(zip(["naive", "HT", "Hajek"], est_shift.std(axis=0, ddof=1).round(4)))
```

Out[8]:
```
{'naive': np.float64(0.0054),
 'HT': np.float64(1.1556),
 'Hajek': np.float64(0.0062)}
```

The price of reweighting: effective sample size

Weighting removes bias but costs precision: a few items with large weights $1/\pi_i$ dominate the estimate. A convenient summary is Kish's **effective sample size**

$$n_{\text{eff}} = \frac{\left(\sum_{i \in S} w_i\right)^2}{\sum_{i \in S} w_i^2} \leq n,$$

i.e. the size of an unweighted sample with the same precision. When $n_{\text{eff}} \ll n$, the π_i are too heterogeneous. Common remedies are **trimming** (capping) the weights, which trades a little bias for a lot of variance, or changing the acquisition itself, for example by stratified crawling.

In [9]:
```
w = 1.0 / pi[included]
n_eff = w.sum()**2 / (w**2).sum()
{"n": int(included.sum()), "n_eff": float(n_eff), "max weight": float(w.max())}
```

Out[9]:
```
{'n': 9106, 'n_eff': 7387.749859202362, 'max weight': 45.78174401114084}
```

When π_i must be guessed

In real acquisition π_i is rarely given; it must be *modelled* (from ranking rules, crawl logs, or a comparison with a small reference sample). Two cases behave very differently:

- π_i known **up to a constant**: the Hájek estimator is unaffected;

- π_i of the **wrong shape**: bias is only partly removed, or even shifted in a new direction.

In [10]:
```
candidates = {
    "true pi":               pi,
    "pi x 4 (wrong scale)":  4 * pi,
    "pi ~ popularity (no intercept)": popularity,
    "constant (= naive)":    np.ones(N),
}
for name, p in candidates.items():
    r = []
    for _ in range(300):
        inc = rng.random(N) < pi                  # data always come from the TR
        r.append((y[inc] / p[inc]).sum() / (1.0 / p[inc]).sum())
    r = np.array(r)
    print(f"{name:32s} bias = {r.mean() - true_mean:+.4f}   sd = {r.std(ddof=1):
```

```
true pi                          bias = -0.0004   sd = 0.0059
pi x 4 (wrong scale)             bias = +0.0001   sd = 0.0062
pi ~ popularity (no intercept)   bias = -0.1129   sd = 0.0125
constant (= naive)               bias = +0.2084   sd = 0.0054
```

Take-aways

- A crawl or an API is a **sampling design**; the naive mean estimates the π-weighted mean, not the population mean.
- Inverse probability weighting (Horvitz–Thompson) removes the bias exactly, provided $\pi_i > 0$ and π_i is known.
- The self-normalised Hájek form needs no N, only relative π_i, is shift-invariant, and usually has far lower variance.
- Weighting costs precision ($n_{\text{eff}} < n$), and a misspecified π leaves residual bias.
- Items with $\pi_i = 0$ are **invisible**: no estimator can recover what the acquisition never had a chance to see.

The practical lesson: **record how each item was selected.** If you cannot even estimate π_i, you cannot know what your dataset represents. This is why provenance is not bureaucracy but part of the statistics.

The same identity $E[I_i/\pi_i] = 1$ underlies importance sampling in Monte Carlo methods and propensity-score weighting in causal inference.

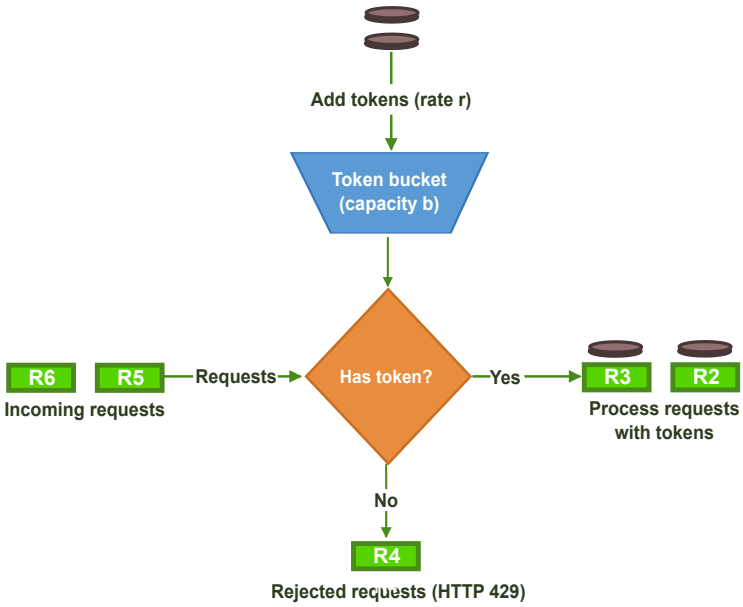
2.2 Rate limits: token bucket and exponential backoff with jitter

The token bucket

Servers meter access. A common model is the **token bucket** with rate r (tokens per second) and capacity b. Its state is the number of tokens $B(t) \in [0, b]$:

- between requests the bucket refills linearly and saturates at b: $B(t) = \min\big(b,\ B(s) + r\,(t - s)\big)$ for $t > s$;

- a request arriving at t is **accepted** if $B(t) \geq 1$, and then $B \leftarrow B - 1$; otherwise it is **refused** (HTTP 429 *Too Many Requests*).



Proposition. Let $A(s, s+\tau)$ be the number of requests accepted in any interval $[s, s+\tau]$. Then

$$A(s, s+\tau) \;\leq\; b + r\,\tau.$$

Proof. At time s the bucket holds at most b tokens. During the interval at most $r\tau$ tokens flow in. Each acceptance removes one token, and B never becomes negative. □

The two parameters therefore have distinct roles:

- r is the **sustained rate**: dividing by τ and letting $\tau \to \infty$, the long-run throughput is at most r;
- b is the **burst size**: at most b requests can be served at a single instant.

The function

$$\alpha(\tau) = b + r\tau$$

is called an *arrival curve* in network calculus.

Insights:

- the refill **saturates**:
  - after b/r seconds of idleness the bucket is full
  - waiting longer buys nothing
- if you know r and b (APIs often publish them, or report them in headers such as `X-RateLimit-Remaining`):
  - **pace** your requests at spacing $\geq 1/r$ and you will never see a 429
  - retrying is what you do when you do not know the limits, or when you share them with other clients

In [11]:
```
class TokenBucket:
    """Server-side rate limiter: rate r tokens/s, capacity b."""
```

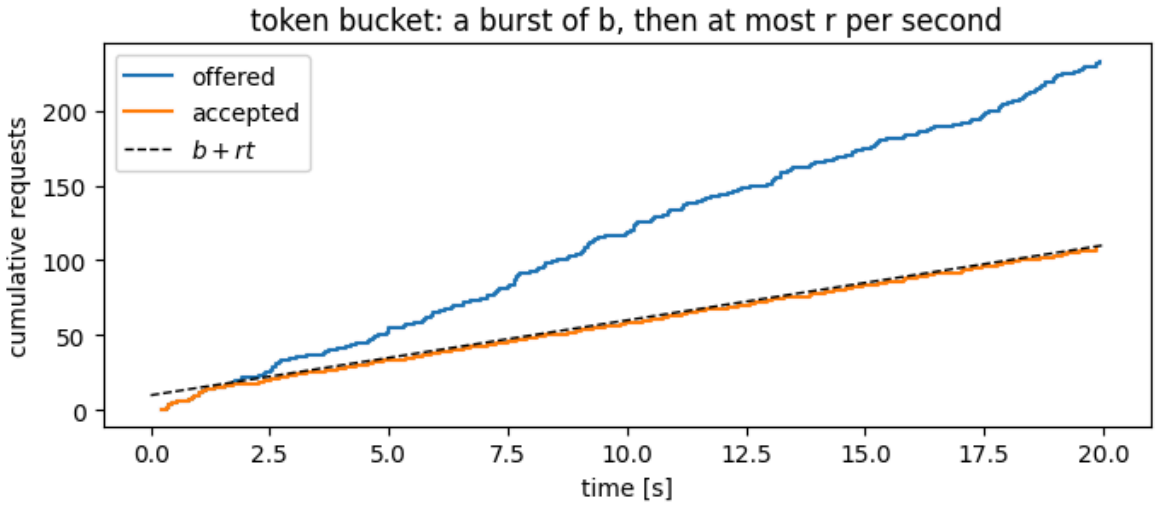
```
    def __init__(self, rate, capacity):
        self.r, self.b = rate, capacity
        self.tokens, self.t = float(capacity), 0.0      # start full
    def allow(self, t):
        self.tokens = min(self.b, self.tokens + self.r * (t - self.t))
        self.t = t
        if self.tokens >= 1.0:
            self.tokens -= 1.0
            return True
        return False

# An impatient client: Poisson requests at rate 12/s against a limit r = 5/s, b
r, b, lam, T = 5.0, 10, 12.0, 20.0
arrivals = np.cumsum(rng.exponential(1 / lam, int(2 * lam * T)))
arrivals = arrivals[arrivals < T]
bucket = TokenBucket(r, b)
ok = np.array([bucket.allow(t) for t in arrivals])
acc = arrivals[ok]

# check the proposition on every interval spanned by two accepted requests
i, j = np.triu_indices(len(acc))
excess = (j - i + 1) - (b + r * (acc[j] - acc[i]))

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.step(arrivals, np.arange(1, len(arrivals) + 1), where="post", label="offered"
ax.step(acc, np.arange(1, len(acc) + 1), where="post", label="accepted")
tt = np.linspace(0, T, 200)
ax.plot(tt, b + r * tt, "k--", lw=1, label="$b + r t$")
ax.set_xlabel("time [s]"); ax.set_ylabel("cumulative requests")
ax.set_title("token bucket: a burst of b, then at most r per second"); ax.legend
plt.tight_layout()
plt.show()

{"offered rate": len(arrivals) / T, "accepted rate": float(ok.sum() / T),
 "max violation of b + r*tau (must be <= 0)": float(excess.max())}
```



Out[11]:
```
{'offered rate': 11.65,
 'accepted rate': 5.4,
 'max violation of b + r*tau (must be <= 0)': -0.007923129884847668}
```

Why retries must be randomised and grow exponentially

Suppose M clients were refused at the same moment and all want to retry. Discretise time into **slots** of length $1/r$, so the server can serve about one request per slot. Each

client picks one of W slots uniformly at random. A client succeeds if nobody else picked its slot:

$$P(\text{alone}) = \left(1 - \frac{1}{W}\right)^{M-1}, \qquad S(W) = \frac{M}{W}\left(1 - \frac{1}{W}\right)^{M-1} \approx x\, e^{-x}, \quad x = \frac{M}{W},$$

where $S(W)$ is the expected number of successes per slot. Two conclusions follow.

1. **Randomness is essential.** Without it every client picks the *same* slot, which is the case $W = 1$, and nobody succeeds (for $M > b$). Retrying deterministically, however patiently, only reproduces the collision.
2. **The window must match the contention.** Maximising xe^{-x} gives $x = 1$: throughput peaks at $W \approx M$ with value $1/e \approx 0.37$. A window too small causes collisions; a window too large wastes slots.

A client does not know M. **Doubling** the window after every failure, $W_k = W_0\, 2^k$, reaches M after about $\log_2(M/W_0)$ failures. The total time spent in the too-small windows is a geometric sum,

$$\sum_{j<k} W_0\, 2^j = W_0\,(2^k - 1) < W_k,$$

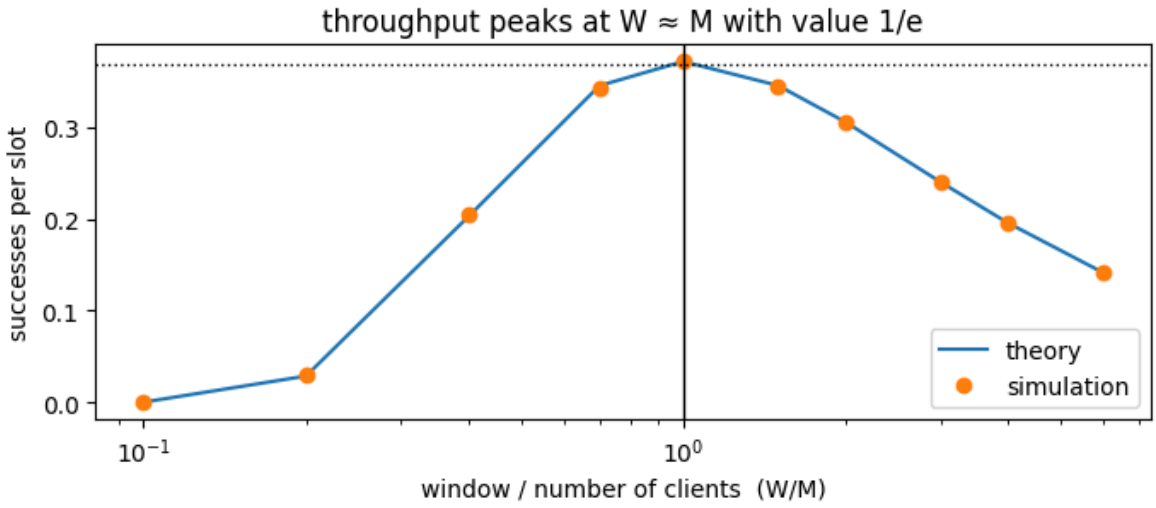
so exponential growth costs at most about a factor of 2 compared with knowing M in advance. A linear schedule $W_k = W_0 k$ would need M/W_0 rounds and total time of order M^2/W_0. This is the argument behind binary exponential backoff in Ethernet (Metcalfe & Boggs, 1976).

In [12]:
```
M = 50
Ws = np.array([5, 10, 20, 35, 50, 75, 100, 150, 200, 300])

def successes_per_slot(M, W, reps=2000, rng=rng):
    slots = rng.integers(0, W, size=(reps, M))
    counts = np.apply_along_axis(np.bincount, 1, slots, minlength=W)
    return (counts == 1).sum(axis=1).mean() / W

sim = [successes_per_slot(M, W) for W in Ws]
theory = (M / Ws) * (1 - 1 / Ws) ** (M - 1)

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(Ws / M, theory, label="theory")
ax.plot(Ws / M, sim, "o", label="simulation")
ax.axvline(1, color="k", lw=1); ax.axhline(np.exp(-1), color="k", lw=1, ls=":")
ax.set_xscale("log"); ax.set_xlabel("window / number of clients  (W/M)")
ax.set_ylabel("successes per slot")
ax.set_title("throughput peaks at W ≈ M with value 1/e")
ax.legend()
plt.tight_layout()
plt.show()
```



Exponential backoff with jitter

Translating the slot model into waiting times: after the k-th failure ($k = 0, 1, \ldots$) define the envelope

$$e_k = \min\big(\text{cap},\ \text{base} \cdot 2^k\big).$$

Here cap is the maximum wait:

- the envelope doubles until $\text{base} \cdot 2^k$ reaches cap (after $k^* = \lceil \log_2(\text{cap}/\text{base}) \rceil$ failures)
- the envelope stays constant afterwards
- it bounds the worst-case latency
- it also bounds the retry window to $\text{cap} \cdot r$ slots

Let us introduce the following wait

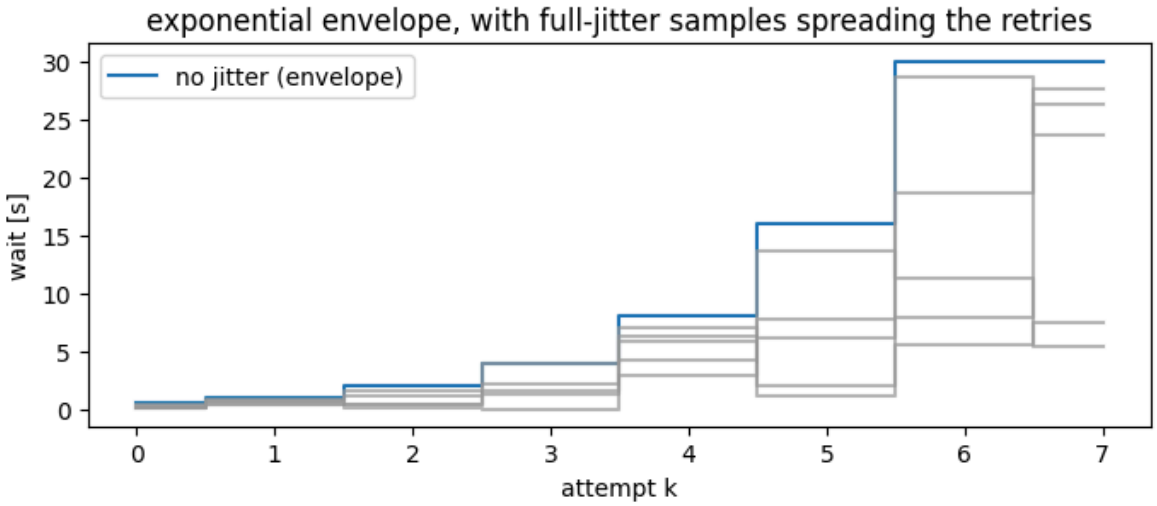
$$w_k^{\text{full}} = U\, e_k, \qquad w_k^{\text{equal}} = \tfrac{1}{2} e_k + \tfrac{1}{2} U\, e_k, \qquad U \sim \text{Uniform}(0, 1).$$

- **Full jitter** is exactly the slot model: a uniform choice within a window that doubles. Its mean wait is $E[w_k] = e_k/2$.
- **Equal jitter** guarantees a minimum pause of $e_k/2$ (mean $3e_k/4$), at the cost of less spreading.
- **No jitter** ($w_k = e_k$) spreads retries in time but not *between clients*: everyone who failed together retries together, forever.
- The **cap** bounds a single wait, and a practical client also limits the number of attempts. If the server sends a `Retry-After` header, it takes precedence over the formula.

In [15]:
```
def backoff_delays(n_attempts, base=0.5, cap=30.0, jitter="full", rng=None):
    rng = np.random.default_rng() if rng is None else rng
    exp = np.minimum(cap, base * 2.0 ** np.arange(n_attempts))
    if jitter == "none":
        return exp
    if jitter == "full":
        return rng.random(n_attempts) * exp
    if jitter == "equal":
```

```
        return exp / 2 + rng.random(n_attempts) * (exp / 2)
    raise ValueError(jitter)

n = 8
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.step(range(n), backoff_delays(n, jitter="none"), where="mid", label="no jitte
for _ in range(5):
    ax.step(range(n), backoff_delays(n, jitter="full", rng=rng), where="mid",
            color="0.6", alpha=0.7)
ax.set_title("exponential envelope, with full-jitter samples spreading the retri
ax.set_xlabel("attempt k"); ax.set_ylabel("wait [s]"); ax.legend()
plt.tight_layout()
plt.show()
```



Thundering herd: what jitter buys

Now we combine both halves. $M = 200$ clients hit a server with $r = 20$/s and $b = 5$ at the same instant, for example after a short outage. Each client retries according to its policy until it succeeds. We count the total number of calls (the load we put on the server) and the time until every client is served.

A lower bound is easy: b clients succeed at once and the remaining $M - b$ at rate r, so no policy can finish before $(M - b)/r \approx 9.8$ s or use fewer than M calls.

Without jitter something worse than "slow" happens. Clients refused together share the same k, so they stay synchronised. Once the waits exceed b/r, the bucket is **full** (saturated at b) when a wave arrives, so every wave admits exactly b clients. We need about $M/b = 40$ waves, most of them spaced by the cap.

In [16]:
```
import heapq

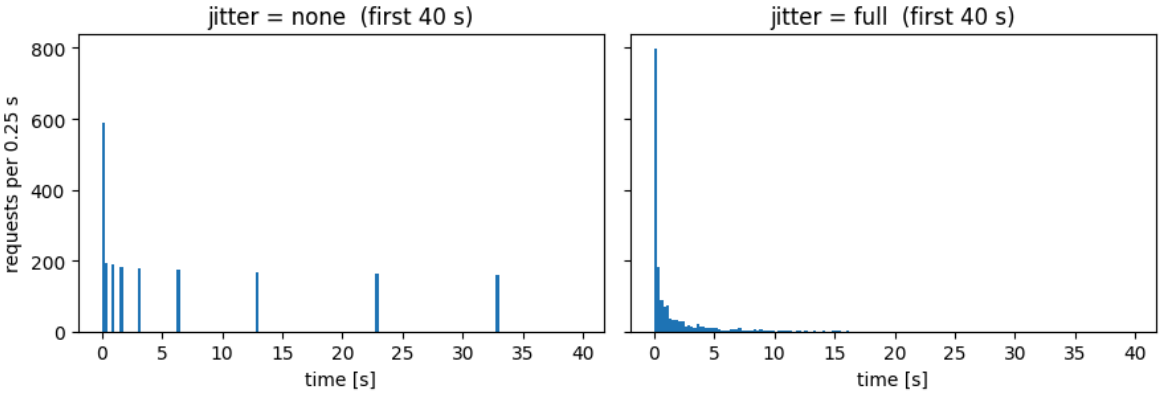
def thundering_herd(M, rate, capacity, jitter, base=0.05, cap=10.0, rng=rng):
    """Event-driven simulation: M clients start at t=0 and retry with backoff un
    bucket = TokenBucket(rate, capacity)
    events = [(0.0, c, 0) for c in range(M)]          # (time, client, failures
    heapq.heapify(events)
    times, t_done = [], 0.0
    while events:
        t, c, k = heapq.heappop(events)
        times.append(t)
```

```
        if bucket.allow(t):
            t_done = t
            continue
        w = backoff_delays(k + 1, base=base, cap=cap, jitter=jitter, rng=rng)[k]
        heapq.heappush(events, (t + w, c, k + 1))
    return np.array(times), t_done

M, rate, capacity = 200, 20.0, 5
results = {}
for jitter in ["none", "equal", "full"]:
    runs = [thundering_herd(M, rate, capacity, jitter) for _ in range(20)]
    results[jitter] = runs
    calls = np.mean([len(tm) for tm, _ in runs])
    t_end = np.mean([td for _, td in runs])
    print(f"{jitter:6s} calls = {calls:6.0f} ({calls / M:4.1f} per client)   all
print(f"bound  calls >= {M}, time >= {(M - capacity) / rate:.1f} s")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.2), sharey=True)
for ax, jitter in zip(axes, ["none", "full"]):
    times, _ = results[jitter][0]
    ax.hist(times, bins=np.arange(0, 40, 0.25))
    ax.set_title(f"jitter = {jitter}  (first 40 s)"); ax.set_xlabel("time [s]")
axes[0].set_ylabel("requests per 0.25 s")
plt.tight_layout()
plt.show()
```

```
none   calls =   4410 (22.1 per client)   all served after  342.8 s
equal  calls =   1570 ( 7.9 per client)   all served after   20.0 s
full   calls =   1623 ( 8.1 per client)   all served after   15.6 s
bound  calls >= 200, time >= 9.8 s
```



Take-aways

- A token bucket admits at most $b + r\tau$ requests in any window of length τ: r limits the rate, b the burst.
- If the limits are known, **pace** requests at spacing $1/r$; backoff is for unknown or shared limits.
- Retries must be **random**: deterministic retries keep colliding clients synchronised, and a saturated bucket then admits only b of them per wave.
- The retry window should scale with the number of competitors; **doubling** finds that scale without knowing it, at a cost of at most a factor of 2.
- Full jitter gives the fastest completion; equal jitter trades a little speed for a guaranteed minimum pause. Both are more than an order of magnitude better than no jitter.

Further reading: M. Brooker, *Exponential Backoff and Jitter*, AWS Architecture Blog (2015); RFC 6585 (status 429) and RFC 9110 (`Retry-After`).

2.3 Deduplication: Jaccard, MinHash, and why it matters

Crawled and scraped corpora are riddled with duplicates and near-duplicates: mirrors, reposts, syndicated news, templates and boilerplate. Duplicates are not harmless redundancy:

- they **inflate frequencies** and bias every estimate, just like the non-uniform inclusion of Section 2.1, because a document present three times counts three times;
- they **waste** storage and training compute;
- most dangerously, they **leak between training and test splits**, so a model is rewarded for memorising rather than generalising. We measure this effect at the end of the section.

Exact duplicates: fingerprints

The easy case is solved by hashing. We map each document to a **fingerprint**, a cryptographic hash of a normalised text, and group documents with equal fingerprints. This costs $O(n)$ for n documents, because a dictionary replaces pairwise comparison. The normalisation step (case, whitespace, Unicode form, stripped markup) is where we *decide* what counts as "the same" document.

In [17]:
```
import hashlib, re

def normalise(text):
    return re.sub(r"\s+", " ", text.lower()).strip()

def fingerprint(text):
    return hashlib.sha256(normalise(text).encode("utf-8")).hexdigest()

docs = ["The quick brown fox jumps over the lazy dog.",
        "the quick   brown fox jumps over the LAZY dog.",
        "The quick brown fox leaps over the lazy dog.",
        "A completely different sentence."]

groups = {}
for i, d in enumerate(docs):
    groups.setdefault(fingerprint(d), []).append(i)
list(groups.values())        # [[0, 1], [2], [3]]: one changed word already defe
```

Out[17]:
```
[[0, 1], [2], [3]]
```

Near-duplicates: shingles and the Jaccard similarity

A cryptographic hash changes completely after a one-character edit, so near-duplicates need a **similarity**. We represent a document by its set of **shingles**, the contiguous fragments of length q cut out by a sliding window:

$$S_q(t) = \{\, t_i t_{i+1} \dots t_{i+q-1} : i = 1, \dots, L - q + 1 \,\}.$$

Shingles can be made of characters (robust to typos, good for short texts) or words (*w-shingling*, standard for long documents). They keep word order *locally*, and a single edit changes at most q of them. The choice of q is a trade-off: for small q every text shares most shingles with every other, while for large q even a small edit destroys many shingles. Typical values are $q = 5$–9 characters or 3–5 words.

For two shingle sets A, B the overlap is the **Jaccard similarity**

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1],$$

with $J = 1$ iff $A = B$ and $J = 0$ iff the sets are disjoint. The **Jaccard distance** $1 - J$ is a proper metric (it satisfies the triangle inequality), so "near-duplicate" means "close in a metric space".

MinHash

Computing J for every pair is $O(n^2)$ in the number of documents, and each evaluation costs time proportional to the set sizes. **MinHash** compresses each set into a short signature whose agreement rate estimates J.

Theorem (Broder, 1997). Let σ be a uniformly random permutation of the universe of shingles. Then

$$P\big[\min \sigma(A) = \min \sigma(B)\big] = J(A, B).$$

Proof. Let x^* be the element of $A \cup B$ with the smallest value $\sigma(x)$. By symmetry of a random permutation, every element of $A \cup B$ is equally likely to be x^*. If $x^* \in A \cap B$, both minima equal $\sigma(x^*)$. If $x^* \in A \setminus B$, then $\min \sigma(A) = \sigma(x^*) < \min \sigma(B)$, because σ is injective, and symmetrically for $x^* \in B \setminus A$. Hence the minima agree iff $x^* \in A \cap B$, which happens with probability $|A \cap B| / |A \cup B|$. □

- $\sigma : U \to \{1, \dots, |U|\}$ is a bijection that assigns every shingle a distinct position in a random order
- $\sigma(A) = \{\sigma(x) : x \in A\}$ is the set of positions of the cards in A
- $\min \sigma(A)$ is the position of the topmost card belonging to A
- "uniformly random" means that all $|U|!$ orderings are equally likely

The estimator. With k independent permutations $\sigma_1, \dots, \sigma_k$, the indicators $Z_j = 1[\min \sigma_j(A) = \min \sigma_j(B)]$ are i.i.d. Bernoulli(J), so

$$\hat{J} = \frac{1}{k} \sum_{j=1}^{k} Z_j, \qquad E[\hat{J}] = J, \qquad \mathrm{Var}(\hat{J}) = \frac{J(1 - J)}{k}.$$

The estimator is **unbiased**, and its accuracy is controlled by k, **not by the size of the sets**. Since $J(1 - J) \le 1/4$, the standard deviation never exceeds $1/(2\sqrt{k})$. Hoeffding's inequality gives a guarantee for every J:

$$P\big(|\hat{J} - J| \ge \varepsilon\big) \le 2\, e^{-2k\varepsilon^2},$$

so error $\varepsilon = 0.1$ with probability at least 95% needs $k \geq \ln(40)/(2 \cdot 0.01) \approx 185$.

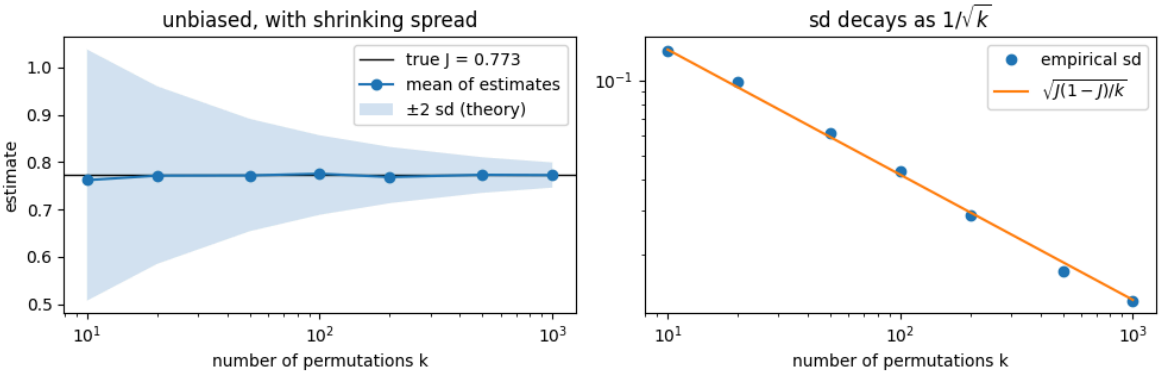
In [18]:
```
def shingles(text, q=3):
    """Set of character q-grams (shingles) of a text."""
    return set(text[i:i + q] for i in range(len(text) - q + 1))

def minhash_estimate(A, B, n_perm, rng=rng):
    """MinHash estimate of J(A, B) with n_perm explicit random permutations."""
    universe = list(A | B)
    index = {s: i for i, s in enumerate(universe)}
    a = np.fromiter((index[s] for s in A), dtype=int)
    b = np.fromiter((index[s] for s in B), dtype=int)
    perms = rng.random((n_perm, len(universe))).argsort(axis=1)   # one permutat
    return np.mean(perms[:, a].min(axis=1) == perms[:, b].min(axis=1))

A = shingles("the quick brown fox jumps over the lazy dog")
B = shingles("the quick brown fox leaps over the lazy dog")
true_J = len(A & B) / len(A | B)

ks = np.array([10, 20, 50, 100, 200, 500, 1000])
R = 300                                                    # repetitions pe
est = np.array([[minhash_estimate(A, B, k) for _ in range(R)] for k in ks])
sd_theory = np.sqrt(true_J * (1 - true_J) / ks)

fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
axes[0].axhline(true_J, color="k", lw=1, label=f"true J = {true_J:.3f}")
axes[0].plot(ks, est.mean(axis=1), "o-", label="mean of estimates")
axes[0].fill_between(ks, true_J - 2 * sd_theory, true_J + 2 * sd_theory, alpha=0
                     label="±2 sd (theory)")
axes[0].set_xscale("log"); axes[0].set_xlabel("number of permutations k")
axes[0].set_ylabel("estimate"); axes[0].set_title("unbiased, with shrinking spre
axes[1].loglog(ks, est.std(axis=1, ddof=1), "o", label="empirical sd")
axes[1].loglog(ks, sd_theory, label=r"$\sqrt{J(1-J)/k}$")
axes[1].set_xlabel("number of permutations k"); axes[1].set_title(r"sd decays as
axes[1].legend()
plt.tight_layout()
plt.show()
```



From permutations to hash functions: signatures

Explicit permutations of the universe are impossible in practice, since the universe of all possible shingles is astronomically large and unknown in advance. Instead we map each shingle to an integer x and use random **hash functions**

$$h_j(x) = (a_j\, x + b_j) \bmod p, \qquad j = 1, \ldots, k,$$

with a large prime p and random $a_j \in \{1, \dots, p-1\}$, $b_j \in \{0, \dots, p-1\}$. Each h_j behaves approximately like a random permutation of $\{0, \dots, p-1\}$, which is close enough for the theorem to hold with negligible error.

The **signature** of a document is the vector of k minima,

$$\operatorname{sig}(A) = \big(\min_{x \in A} h_1(x),\ \dots,\ \min_{x \in A} h_k(x)\big),$$

and the fraction of coordinates on which two signatures agree estimates J. The decisive gain is that signatures are computed **once per document**. Afterwards every document is a fixed vector of k integers, whatever its length, and comparing two documents costs $O(k)$. Comparing *all* pairs is still $O(n^2)$; the banding trick of **locality-sensitive hashing** (Week 13 or 14) removes that last obstacle.

We test this on a synthetic corpus of 300 random documents plus 20 planted near-duplicates, each a copy of one document with 1–7 words replaced.

In [19]:
```
import zlib

P = (1 << 31) - 1                                  # Mersenne prime 2^31 - 1

def make_hashes(n_hashes, seed=0):
    r = np.random.default_rng(seed)
    a = r.integers(1, P, n_hashes, dtype=np.int64)
    b = r.integers(0, P, n_hashes, dtype=np.int64)
    return a, b

def signature(shingle_set, a, b):
    # crc32 gives a deterministic integer id per shingle (Python's hash() is sal
    x = np.fromiter((zlib.crc32(s.encode("utf-8")) % P for s in shingle_set), dt
    return ((np.outer(a, x) + b[:, None]) % P).min(axis=1)      # a*x < 2^62: no

# synthetic corpus with planted near-duplicates
vocab = np.array([f"w{i}" for i in range(500)])

def random_doc(n_words=40):
    return " ".join(rng.choice(vocab, n_words))

def perturb(doc, n_edits):
    words = doc.split()
    for _ in range(n_edits):
        words[rng.integers(len(words))] = rng.choice(vocab)
    return " ".join(words)

n_base, n_dup = 300, 20
corpus = [random_doc() for _ in range(n_base)]
planted = set()
for j in range(n_dup):
    corpus.append(perturb(corpus[j], n_edits=int(rng.integers(1, 8))))
    planted.add((j, n_base + j))

a, b = make_hashes(128)
shs = [shingles(d, q=5) for d in corpus]
sigs = np.array([signature(s, a, b) for s in shs])            # (n_docs, k) inte

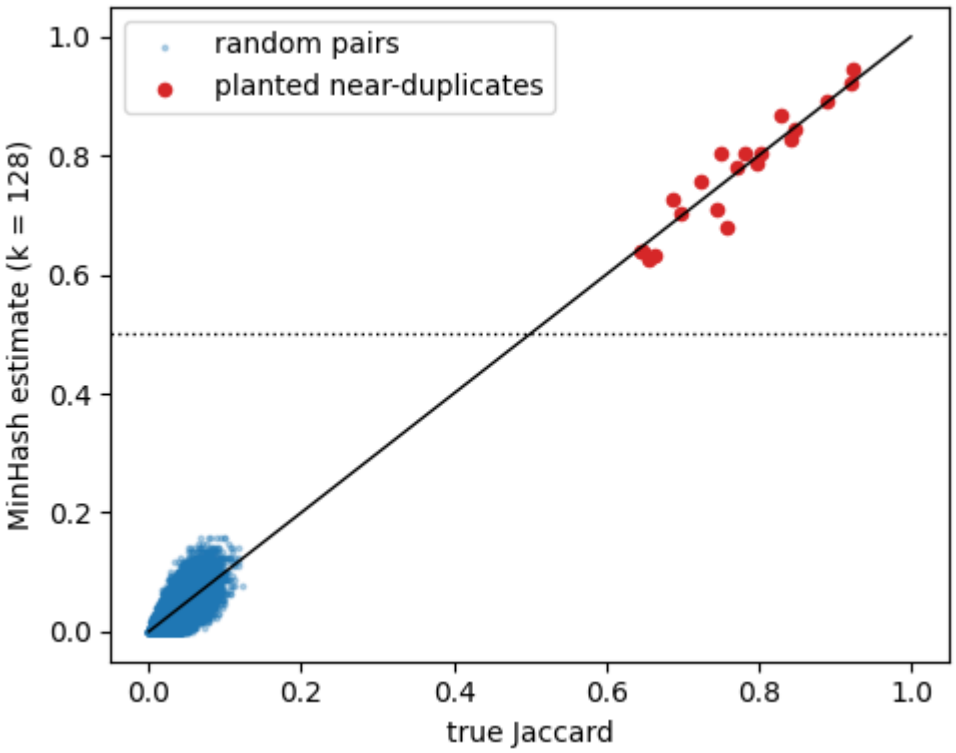
iu, ju = np.triu_indices(len(corpus), k=1)
```

```
J_est = (sigs[iu] == sigs[ju]).mean(axis=1)                    # all pairs, from
J_true = np.array([len(shs[i] & shs[j]) / len(shs[i] | shs[j]) for i, j in zip(i

threshold = 0.5
found = {(int(i), int(j)) for i, j in zip(iu[J_est > threshold], ju[J_est > thre
is_planted = np.array([(int(i), int(j)) in planted for i, j in zip(iu, ju)])

fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(J_true[~is_planted], J_est[~is_planted], s=3, alpha=0.3, label="rando
ax.scatter(J_true[is_planted], J_est[is_planted], s=20, color="C3", label="plant
ax.plot([0, 1], [0, 1], "k", lw=1); ax.axhline(threshold, color="k", ls=":", lw=
ax.set_xlabel("true Jaccard"); ax.set_ylabel("MinHash estimate (k = 128)"); ax.l
plt.tight_layout()
plt.show()

{"pairs compared": len(J_est), "flagged": len(found),
 "planted recovered": len(found & planted), "false positives": len(found - plant
 "sd of estimation error": float(np.std(J_est - J_true))}
```



Out[19]:
```
{'pairs compared': 51040,
 'flagged': 20,
 'planted recovered': 20,
 'false positives': 0,
 'sd of estimation error': 0.017006586025243346}
```

Why it matters: duplicates leak across splits

The simplest demonstration uses labels that are **pure noise**, so no model can beat 50% accuracy on genuinely new data. We take 1000 random points, add near-copies of 300 of them (with the same label), and evaluate a 1-nearest-neighbour classifier under two splits:

- a **random split**, which ignores duplicates;
- a **group split**, which keeps each original together with its copies on the same side.

Expected accuracy of the random split: about $600/1300 \approx 46\%$ of the test points belong to a duplicate pair, and in $\approx 70\%$ of these cases the twin landed in the training set. These $\approx 32\%$ of test points are classified correctly by memory, the rest at chance: $0.32 + 0.68 \cdot 0.5 \approx 0.66$.

In [20]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, GroupShuffleSplit

n, D = 1000, 20
X_leak = rng.standard_normal((n, D))
y_leak = rng.integers(0, 2, n)                      # labels are pure noise
group = np.arange(n)
src = rng.choice(n, 300, replace=False)             # 300 items get a near-copy
X_leak = np.vstack([X_leak, X_leak[src] + 0.05 * rng.standard_normal((len(src),
y_leak = np.concatenate([y_leak, y_leak[src]])
group = np.concatenate([group, group[src]])         # a copy shares the group id

def accuracy(train, test):
    model = KNeighborsClassifier(n_neighbors=1).fit(X_leak[train], y_leak[train]
    return model.score(X_leak[test], y_leak[test])

scores = {"random split": [], "group split": []}
for seed in range(50):
    tr, te = train_test_split(np.arange(len(y_leak)), test_size=0.3, random_stat
    scores["random split"].append(accuracy(tr, te))
    tr, te = next(GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=seed
                  .split(X_leak, y_leak, group))
    scores["group split"].append(accuracy(tr, te))

{k: f"{np.mean(v):.3f} ± {np.std(v):.3f}" for k, v in scores.items()}
```

Out[20]:
```
{'random split': '0.651 ± 0.021', 'group split': '0.490 ± 0.031'}
```

Take-aways

- Exact duplicates are found in $O(n)$ by fingerprints; the normalisation decides what "the same" means.
- Near-duplicates need a similarity: shingle sets with the Jaccard similarity, whose distance $1 - J$ is a metric.
- MinHash turns J into a probability: $P[\min \sigma(A) = \min \sigma(B)] = J$. Averaging k hashes gives an unbiased estimate with standard deviation at most $1/(2\sqrt{k})$, independent of document length.
- Signatures are computed once per document and compared in $O(k)$; LSH avoids comparing all pairs.
- Duplicates shared between training and test sets inflate scores even on pure noise. Deduplicate *before* splitting, or split by groups of near-duplicates.

Further reading: A. Broder, *On the resemblance and containment of documents* (1997); Leskovec, Rajaraman & Ullman, *Mining of Massive Datasets*, ch. 3 (free online).

2.4 Near-duplicate search at scale — the device pattern again

Exact and MinHash dedup handle text sets. When documents are already **vectors** (embeddings), near-duplicates are pairs with cosine similarity above a threshold, and the whole comparison is one matrix product $X_{\text{norm}} X_{\text{norm}}^{\top}$ — exactly the parallel workload the CPU / CUDA / MPS pattern from Week 1 is for. We first state the reference in NumPy, then run the same computation on whichever device is available.

In [21]:
```
def near_duplicates_numpy(X, threshold=0.98):
    Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
    S = Xn @ Xn.T
    np.fill_diagonal(S, 0.0)
    return {tuple(sorted(p)) for p in np.argwhere(S > threshold)}

# Synthetic corpus of 800 vectors with 5 planted near-duplicate pairs.
D, n = 256, 800
X = rng.standard_normal((n, D)).astype(np.float32)
for j in range(5):
    X[400 + j] = X[j] + 0.01 * rng.standard_normal(D).astype(np.float32)

sorted(near_duplicates_numpy(X))
```

Out[21]:
```
[(np.int64(0), np.int64(400)),
 (np.int64(1), np.int64(401)),
 (np.int64(2), np.int64(402)),
 (np.int64(3), np.int64(403)),
 (np.int64(4), np.int64(404))]
```

In [22]:
```
import torch

def get_device():                          # same helper as Week 1
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")

def near_duplicates(X, threshold=0.98, device=None):
    device = get_device() if device is None else device
    Xt = torch.as_tensor(X, dtype=torch.float32, device=device)
    Xn = Xt / Xt.norm(dim=1, keepdim=True)
    S = Xn @ Xn.T
    S.fill_diagonal_(0.0)
    idx = torch.argwhere(S > threshold).cpu().numpy()
    return {tuple(sorted(p)) for p in idx}

device = get_device()
found = near_duplicates(X, device=device)
assert found == near_duplicates_numpy(X)     # device result matches the referen
{"device": str(device), "pairs": sorted(found)}
```

Out[22]:
```
{'device': 'cpu',
 'pairs': [(np.int64(0), np.int64(400)),
  (np.int64(1), np.int64(401)),
  (np.int64(2), np.int64(402)),
  (np.int64(3), np.int64(403)),
  (np.int64(4), np.int64(404))]}
```

For a corpus too large to fit an $n \times n$ matrix in memory, the same product is computed in blocks of rows, or replaced by an approximate-nearest-neighbour index (FAISS) — the

Week 13 (or 14) topic. The point here is that *deduplication is a modelling step*, not clean-up: it decides what "one example" means.

3. Sources in practice

3.1 Public repositories

Reach for a curated repository before building a scraper — the data is versioned, licensed and citable. Common starting points:

- **Hugging Face Datasets** — text, image and audio, streaming-friendly.
- **OpenML / UCI** — classic tabular and benchmark sets.
- **Kaggle Datasets** — competitions and community data.
- **Zenodo** — DOI-citable research artefacts.
- **Wikimedia dumps / Common Crawl** — web- and encyclopaedia-scale corpora.

The ergonomics are all similar. A dataset bundled with scikit-learn stands in for the pattern so this runs offline:

In [23]:
```
from sklearn.datasets import load_digits

digits = load_digits()
X_img, y_img = digits.images, digits.target
{"n_samples": int(X_img.shape[0]), "image_shape": tuple(X_img.shape[1:]),
 "classes": sorted(set(y_img.tolist()))}
```

Out[23]:
```
{'n_samples': 1797,
 'image_shape': (8, 8),
 'classes': [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]}
```

3.2 REST APIs

An API returns structured responses but imposes:

- **authentication** (API keys or OAuth),
- **pagination** (results arrive in pages, linked by a cursor or page number),
- **rate limits** (Section 2.2),
- **terms of service** that constrain storage and redistribution.

The collection loop below is the real pattern — request a page, follow the cursor, back off on failure — exercised against a synthetic paginated endpoint so it runs offline. Swapping `mock_api` for a `requests.get` against a live URL is the only change needed.

In [24]:
```
import time

def mock_api(cursor=0, page_size=25, total=90, fail_prob=0.3, rng=None):
    # Stand-in for a paginated endpoint that occasionally returns 429.
    rng = np.random.default_rng() if rng is None else rng
    if rng.random() < fail_prob:
        return {"status": 429, "items": None, "next": cursor}
```

```
    items = [{"id": i, "title": f"record-{i}"}
             for i in range(cursor, min(cursor + page_size, total))]
    nxt = cursor + page_size if cursor + page_size < total else None
    return {"status": 200, "items": items, "next": nxt}

def collect_all(fetch, max_retries=6, base=0.2, cap=5.0, rng=None):
    rng = np.random.default_rng(0) if rng is None else rng
    out, cursor, attempt = [], 0, 0
    while True:
        resp = fetch(cursor=cursor, rng=rng)
        if resp["status"] == 429:
            if attempt >= max_retries:
                raise RuntimeError("giving up after repeated rate limiting")
            delay = min(cap, base * 2.0 ** attempt) * rng.random()
            time.sleep(delay)          # real wait; tiny here
            attempt += 1
            continue
        attempt = 0
        out.extend(resp["items"])
        if resp["next"] is None:
            return out
        cursor = resp["next"]

collected = collect_all(mock_api)
{"n_collected": len(collected), "first": collected[0], "last": collected[-1]}
```

Out[24]:
```
{'n_collected': 90,
 'first': {'id': 0, 'title': 'record-0'},
 'last': {'id': 89, 'title': 'record-89'}}
```

3.3 Web scraping — only when there is no API, and only politely

Scraping is a last resort with obligations. Before requesting a page, consult the site's `robots.txt`, honour any crawl delay, identify your client with a real `User-Agent`, and stay within the terms of service and copyright. Pages rendered by JavaScript need a headless browser (Playwright, Selenium) rather than a plain HTTP request.

In [25]:
```
from urllib.robotparser import RobotFileParser

# A site's robots policy (normally fetched from https://site/robots.txt).
robots_txt = ["User-agent: *", "Disallow: /private/", "Allow: /", "Crawl-delay:
rp = RobotFileParser()
rp.parse(robots_txt)

targets = ["https://example.org/catalog/page1", "https://example.org/private/ord
{url: rp.can_fetch("*", url) for url in targets}
```

Out[25]:
```
{'https://example.org/catalog/page1': True,
 'https://example.org/private/orders': False}
```

3.4 OCR — turning images of text into text

When text exists only as pixels (scanned documents, photographs of signs), OCR recovers it. The pipeline is an image task — load, convert to grayscale, threshold or denoise —

followed by a recogniser such as Tesseract, which is a natural bridge to the image block (Block C). We synthesise a "scan" with a known string so we can check the result exactly.

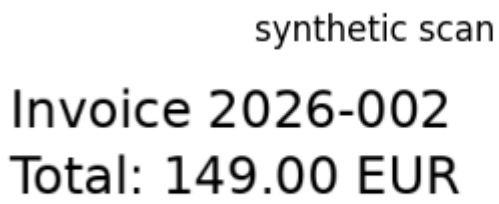
In [28]:
```
from PIL import Image, ImageDraw, ImageFont
from matplotlib import font_manager

# A cross-platform TrueType font shipped with matplotlib.
font = ImageFont.truetype(font_manager.findfont("DejaVu Sans"), 34)

message = "Invoice 2026-002\nTotal: 149.00 EUR"
canvas = Image.new("L", (520, 140), color=255)
ImageDraw.Draw(canvas).multiline_text((16, 16), message, fill=0, font=font, spac

fig, ax = plt.subplots(figsize=(5, 1.6))
ax.imshow(canvas, cmap="gray"); ax.axis("off"); ax.set_title("synthetic scan")
plt.show()

try:
    import pytesseract
    recovered = pytesseract.image_to_string(canvas).strip()
except (ImportError, EnvironmentError) as e:
    recovered = f"(tesseract not installed: {e})"
recovered
```



Out[28]:
```
'Invoice 2026-002\nTotal: 149.00 EUR'
```

On a Mac the recogniser needs the system binary once (`brew install tesseract`); the `try` block keeps the notebook running even where it is absent. Real scans need the preprocessing — deskewing, adaptive thresholding — that we study properly in the third part of the semester.

4. Licensing, PII, provenance and caching

Acquisition ends not with bytes on disk but with data you can **use, trust and cite**.

Licensing. Data and code carry separate licences. Creative-Commons variants (CC0, CC BY, CC BY-SA, CC BY-NC), dataset-specific terms and API terms of service all differ; `NC` forbids commercial use, `SA` forces the same licence downstream. Record the licence with the data.

PII / GDPR (*Personally Identifiable Information/General Data Protection Regulation*) Under EU law, personal data must be minimised and protected. A regex pass removes the obvious cases but is only a first line — it both over- and under-matches, which is exactly why serious pipelines use the named-entity methods.

In [29]:
```
import re

def redact(text):
    text = re.sub(r"[\w.+-]+@[\w-]+\.[\w.-]+", "[EMAIL]", text)
    text = re.sub(r"\+?\d[\d ()-]{7,}\d", "[PHONE]", text)
    return text

sample = "Reach Jan at jan.kowalski@example.com or +48 601 234 567 before Friday
redact(sample)
```

Out[29]:
```
'Reach Jan at [EMAIL] or [PHONE] before Friday.'
```

Provenance and caching. Reproducibility means being able to say exactly where each record came from. A small **manifest** — source, query, timestamp, licence, and a content hash — makes an acquisition auditable, and the same hash doubles as a **content-addressed cache key** so you never re-hit an API for data you already have.

In [30]:
```
import hashlib, json, datetime

def content_hash(obj):
    blob = json.dumps(obj, sort_keys=True).encode("utf-8")
    return hashlib.sha256(blob).hexdigest()

def manifest(source, query, license, records):
    return {"source": source,
            "query": query,
            "retrieved_at": datetime.datetime(2026, 2, 1, 9, 0).isoformat(),
            "license": license,
            "n_records": len(records),
            "content_sha256": content_hash(records)}

# A tiny content-addressed cache: the hash is both the fingerprint and the key.
cache = {}
def get_cached(key_obj, produce):
    key = content_hash(key_obj)
    if key not in cache:
        cache[key] = produce()
    return cache[key], key

data, key = get_cached({"api": "mock", "query": "sensors"}, lambda: collected)
manifest("mock_api", {"query": "sensors"}, "CC-BY-4.0", data)
```

Out[30]:
```
{'source': 'mock_api',
 'query': {'query': 'sensors'},
 'retrieved_at': '2026-02-01T09:00:00',
 'license': 'CC-BY-4.0',
 'n_records': 90,
 'content_sha256': 'a3d360e8fc37a5865d4d2542e5d07b7de4e60eff5fc21d67bcc7f697050
f9420'}
```

Summary

- Acquisition is the first modelling decision: it sets coverage, bias, legality and reproducibility.

- It is a **sampling** process — non-uniform inclusion biases estimates, and inverse-probability weighting corrects it *when you recorded how you sampled*.
- Servers must be treated politely: **token buckets** meter access and **exponential backoff with jitter** retries without stampeding.
- **Deduplication** (Jaccard / MinHash, and vectorised near-duplicate search on device) decides what counts as one example.
- Repositories, APIs, scraping and OCR are four routes to the same goal, each with its own licence, limits and etiquette; **provenance and caching** make the result trustworthy and repeatable.

In [ ]: